11. Python Program to Solve the 8-Queen Problem

Aim

To write a Python program to solve the 8-Queen problem by placing 8 queens on a chessboard so that no two queens attack each other.

Algorithm

1. Start the program.
2. Create an 8 × 8 chessboard.
3. Place one queen in each row.
4. Check whether the queen conflicts with any previous queen.
5. If safe, place the queen and move to the next row.
6. If no safe position exists, backtrack.
7. Display the solution.
8. Stop the program.

Code

```
# 8-Queen Problem using Backtracking

N = 8
board = [-1] * N

def is_safe(row, col):
    for r in range(row):
        if board[r] == col:
            return False
        if abs(board[r] - col) == abs(r - row):
            return False
    return True

def solve(row):
    if row == N:
        return True
```

```
    for col in range(N):
        if is_safe(row, col):
            board[row] = col

            if solve(row + 1):
                return True

            board[row] = -1

    return False

if solve(0):
    print("Solution:")
    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution")
```

OUTPUT

```
                print(".", end=" ")
        print()
else:
    print("No solution")
```

```
Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

12. Python Program for the Water Jug Problem

Aim

To write a Python program to solve the Water Jug problem using the Breadth-First Search (BFS) algorithm.

Algorithm

1. Start the program.
2. Define two jugs with capacities of 4 litres and 3 litres.
3. Set the target amount to 2 litres.
4. Generate possible operations: fill, empty, and pour water.
5. Use BFS to search for the target state.
6. Display the steps to reach the target.
7. Stop the program.

Code

```
# Water Jug Problem using BFS

from collections import deque

def water_jug():
    capacity1 = 4
    capacity2 = 3
    target = 2
```

```
start = (0, 0)
queue = deque([(start, [])])
visited = set()

while queue:
    (a, b), path = queue.popleft()

    if (a, b) in visited:
        continue
    visited.add((a, b))

    new_path = path + [(a, b)]

    if a == target or b == target:
        return new_path

    pour = min(a, capacity2 - b)
    states = [
        (capacity1, b),
        (a, capacity2),
        (0, b),
        (a, 0),
        (a - pour, b + pour)
    ]

    pour = min(b, capacity1 - a)
    states.append((a + pour, b - pour))

    for state in states:
        if state not in visited:
```

```
            queue.append((state, new_path))
return None

solution = water_jug()

if solution:
    print("Steps:")
    for step in solution:
        print(step)
else:
    print("No solution found")
```

OUTPUT:

```
            print(step)
    else:
        print("No solution found")
```

```
Steps:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

13. Python Program for the Crypt-Arithmetic Problem

Aim

To write a Python program to solve the crypt-arithmetic problem SEND + MORE = MONEY, where each letter represents a unique digit.

Algorithm

1. Start the program.
2. Assign different digits to the letters.
3. Ensure that S and M are not zero.
4. Check whether SEND + MORE = MONEY.
5. Display the valid solution.
6. Stop the program.

Code

```
# Crypt-Arithmetic Problem

from itertools import permutations

letters = "SENDMORY"

for digits in permutations(range(10), len(letters)):
    d = dict(zip(letters, digits))

    if d['S'] == 0 or d['M'] == 0:
        continue

    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    MONEY = (10000*d['M'] + 1000*d['O'] +
            100*d['N'] + 10*d['E'] + d['Y'])

    if SEND + MORE == MONEY:
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        break
```

OUTPUT

```
                100*d['N'] + 10*d['E'] + d['Y'])

        if SEND + MORE == MONEY:
            print("SEND =", SEND)
            print("MORE =", MORE)
            print("MONEY =", MONEY)
            break
```

```
...  SEND = 9567
     MORE = 1085
     MONEY = 10652
```

14. Python Program for the Missionaries and Cannibals Problem

Aim

To write a Python program to solve the Missionaries and Cannibals problem using the Breadth-First Search (BFS) algorithm.

Algorithm

1. Start the program.
2. Consider 3 missionaries and 3 cannibals on the left bank.
3. Allow a boat to carry 1 or 2 people.
4. Generate possible boat movements.
5. Ensure cannibals never outnumber missionaries on either bank when missionaries are present.
6. Use BFS to find a valid solution.
7. Display the steps.

Stop the program.

```
# Missionaries and Cannibals using BFS


from collections import deque


def valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False


    # Missionaries must not be outnumbered
    if m > 0 and m < c:
```

```
        return False

    right_m = 3 - m
    right_c = 3 - c

    if right_m > 0 and right_m < right_c:
        return False

    return True

def solve():
    start = (3, 3, 0)  # 0 = left, 1 = right
    goal = (0, 0, 1)

    moves = [(1, 0), (2, 0), (0, 1),
             (0, 2), (1, 1)]

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()
        m, c, boat = state

        if (m, c) == (0, 0):
            return path

        for dm, dc in moves:
```

8.

Code

```
            if boat == 0:
                new_state = (m-dm, c-dc, 1)
            else:
                new_state = (m+dm, c+dc, 0)

            nm, nc, _ = new_state

            if valid(nm, nc) and new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [new_state]))

    return None

solution = solve()

if solution:
    for step, state in enumerate(solution):
        m, c, boat = state
        print("Step", step, ": Left =", m, "M", c, "C",
              "| Right =", 3-m, "M", 3-c, "C",
              "| Boat =", "Left" if boat == 0 else "Right")
else:
    print("No solution found")
```

OUTPUT:

```
        m, c, boat = state
        print("Step", step, ": Left =", m, "M", c, "C",
              "| Right =", 3-m, "M", 3-c, "C",
              "| Boat =", "Left" if boat == 0 else "Right")
else:
    print("No solution found")
```

```
Step 0 : Left = 3 M 3 C | Right = 0 M 0 C | Boat = Left
Step 1 : Left = 3 M 1 C | Right = 0 M 2 C | Boat = Right
Step 2 : Left = 3 M 2 C | Right = 0 M 1 C | Boat = Left
Step 3 : Left = 3 M 0 C | Right = 0 M 3 C | Boat = Right
Step 4 : Left = 3 M 1 C | Right = 0 M 2 C | Boat = Left
Step 5 : Left = 1 M 1 C | Right = 2 M 2 C | Boat = Right
Step 6 : Left = 2 M 2 C | Right = 1 M 1 C | Boat = Left
Step 7 : Left = 0 M 2 C | Right = 3 M 1 C | Boat = Right
Step 8 : Left = 0 M 3 C | Right = 3 M 0 C | Boat = Left
Step 9 : Left = 0 M 1 C | Right = 3 M 2 C | Boat = Right
Step 10 : Left = 1 M 1 C | Right = 2 M 2 C | Boat = Left
Step 11 : Left = 0 M 0 C | Right = 3 M 3 C | Boat = Right
```

15. Python Program for the Vacuum Cleaner Problem

Aim

To write a Python program to simulate a vacuum cleaner that cleans dirty rooms using simple condition-based actions.

Algorithm

1. Start the program.
2. Define two rooms, A and B.
3. Mark each room as clean or dirty.
4. Check the current room.
5. If it is dirty, clean it.
6. Move to the next room and repeat.
7. Display the final room status.
8. Stop the program.

Code

```
# Vacuum Cleaner Problem

rooms = {
   'A': 'Dirty',
   'B': 'Dirty'
```

```
}

location = 'A'

while True:
    print("Vacuum cleaner is in room", location)

    if rooms[location] == 'Dirty':
        rooms[location] = 'Clean'
        print("Room", location, "cleaned")

    if location == 'A':
        location = 'B'
    else:
        break

print("\nFinal Room Status:")
for room, status in rooms.items():
    print("Room", room, ":", status)
```

OUTPUT:

```
            location = 'B'
        else:
            break

    print("\nFinal Room Status:")
    for room, status in rooms.items():
        print("Room", room, ":", status)
```

```
Vacuum cleaner is in room A
Room A cleaned
Vacuum cleaner is in room B
Room B cleaned

Final Room Status:
Room A : Clean
Room B : Clean
```